NLU course project

Alessio Novel (256172)

University of Trento

alessio.novel@studenti.unitn.it

The following is the report for the second part of the project: the one about Natural Language Understanding.

1. Introduction

This part of the project focused on implementing and improving a sequence labeling (slot filling) task and a text classification (intent classification) task. Performances were evaluated with F1 score (for the first) and accuracy (for the second).

When tuning the hyperparameters, I decided to keep fixed the following parameters:

- **Patience:** I changed it from 3 to 10 and it stopped successfully the training when the model started to overfit.
- **Batch size:** I decided to keep it constant to avoid increasing computational cost and to focus the comparison on the other hyperparameters.
- **Clip:** I did not see the need to change it.
- **Number of epochs:** In most cases early stopping was activated. In instances where it did not, I interpreted it as a sign of bad training behavior, so I changed other parameters.

I also did not modify the number of layers, bidirectionality (unless requested), and neither added additional regularization techniques nor optimizer techniques. Since in the previous parts I used AdamW, I also make an experiment with that instead of standard Adam.

2. Implementation details

2.1. Part A

I implemented the LSTM model based on lab materials, to use it as Baseline. Here, I noticed that the best model was judged only on the F1 score (slot filling). Since the objective was to train a joint model, I decided to change this and take the model with the best sum of the two losses. For the default configuration, this brings slightly better results also for test F1, probably acting as a form of regularization.

For the first step, I set to true the bidirectional parameter of the LSTM [1]. As shown in the documentation, I also had to concatenate the output of the two directions and then double the size of the linear layers in order to make them able to process the new LSTM output.

For the second step, I implemented the two dropout layers [2], calling the dropout right after the embedding layer and right before the last linear layers. I use the same implementation I've done for the previous part.

Finally, I tried to change the optimizer from Adam to AdamW [3].

2.2. Part B

Here, I tried to implement the finetuning of the pre-trained BERT-base-uncased model [4], trying to change as little as possible the structure I built for the part A since the task was the same.

All the following observations are based on the BERT paper [5].

In this case no initialization is necessary since we start already from the pre-trained model.

Also, it is not necessary to pack the sequence in order to get correctly the last hidden state because here a special [CLS] token is used for that scope.

It is even not necessary to order sequences by length, because the transformer does not take advantage of this like LSTM. The length of the sentences was set to maximum 50 (with padding if smaller) because, as written in the paper, the ATIS dataset doesn't have sentences longer than that value.

Since BERT is pre-trained using sub-word tokens, I have to tokenize the words in this way too. To fit this (as the paper suggests) we assign the label only to the first sub-token of the word, and assign an ignore index to all the other subtokens. In this way all the tokens are considered during attention, but the training and evaluation are guided only by the first sub-word tokens.

3. Results

3.1. Part A

For the baseline I found that increasing the hidden size brings also an increase in performances, until a point in which the model overfits too much. I also tried to decrease the learning rate and I reached better results but with many more epochs.

With bidirectionality the model achieves better performances with less epochs, probably because in this case the model can analyze more dependencies between words. Here as well, decreasing the learning rate led to better results.

Dropout applied correctly a regularization, leading to best performance at all. As for the first part of the project, I can increase the hidden size by decreasing also the dropout rate, reaching better performance each time, until a point in which dropout rate is too high for the model to learn something.

AdamW did not lead to significantly better performances, increasing F1 but decreasing accuracy.

In Table 1 I show the best results I obtained for each configuration.

3.2. Part B

In the finetuning of BERT I first tried the same configuration of Part B, but I found that the learning rate was too aggressive for a model that is already at a good point. With very low training rate I reached the best results of all the experiment. In this case I could not modify the size (which is already very big) so I could only test the learning rate. Also in this case, increasing the dropout leads to better performances. Here as well, I tried using AdamW but it did not improve the performance.

In Table 2 I show the best results I obtained for each configuration.

Table 1: *Best results for each configuration (Part A).*

Configuration	LR	Hidden Size	Dropout	Test F1	Test Intent Acc
Baseline LSTM	0.001	600	-	0.9328	0.9261
Bidirectional LSTM	0.0005	600	-	0.9461	0.9485
Dropout LSTM	0.0005	800	0.7 / 0.8	0.9495	**0.9619**
Dropout LSTM AdamW	0.0005	800	0.6 / 0.7	**0.9510**	0.9597

Table 2: *Best results for each optimizer configuration (Part B).*

Configuration	LR	Dropout	Test F1	Test Intent Acc
BERT Adam	5×10^{-5}	0.6	**0.9580**	**0.9776**
BERT AdamW	2×10^{-5}	0.6	0.9548	**0.9776**

4. Other experiments

In the folders you can find a file named `experiments_results.json` in which there are all the configurations I attempted in the order I did that shows my reasoning during the finetuning part.

5. References

[1] PyTorch, "Long short-term memory (LSTM)," https://docs.pytorch.org/docs/stable/generated/torch.nn.LSTM.html.

[2] ——, "Dropout," https://docs.pytorch.org/docs/stable/generated/torch.nn.Dropout.html.

[3] ——, "Adamw," https://docs.pytorch.org/docs/stable/generated/torch.nn.AdamW.html.

[4] Q. Chen, Z. Zhuo, and W. Wang, "Bert for joint intent classification and slot filling," *arXiv preprint arXiv:1902.10909*, 2019.

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2019.